

WILP, BITS PILANI DEEP NEURAL NETWORKS (AIML ZG511)

Session 12: Transformers, Positional Encoding & BERT

Scaled Dot-Product Attention, Sinusoidal Embeddings, Layer Normalization, and BERT Pre-training

Module 9

Session 12 Reader
Full Worked Solutions

1 The Transformer Architecture & Self-Attention

Transformers eliminate recurrence completely, enabling parallel processing of entire sequences via Self-Attention.

The Transformer model replaces sequential RNN loops with Scaled Dot-Product Self-Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where Q, K, V represent Query, Key, and Value projections.

Full Worked Self-Attention Numerical

Query token 1: $q_1 = [0.21, 0.04, 0.63, 0.36]^T$. Key vectors k_1, k_2, k_3 :

$$k_1 = \begin{bmatrix} 0.31 \\ 0.84 \\ 0.12 \\ 0.45 \end{bmatrix}, \quad k_2 = \begin{bmatrix} 0.15 \\ 0.22 \\ 0.78 \\ 0.09 \end{bmatrix}, \quad k_3 = \begin{bmatrix} 0.55 \\ 0.01 \\ 0.34 \\ 0.88 \end{bmatrix}$$

Key dimension $d_k = 4$, so scaling factor $\sqrt{d_k} = \sqrt{4} = 2.0$.

Step 1. Compute raw dot products $q_1 \cdot k_j$:

$$q_1 \cdot k_1 = 0.0651 + 0.0336 + 0.0756 + 0.1620 = 0.3363$$

$$q_1 \cdot k_2 = 0.0315 + 0.0088 + 0.4914 + 0.0324 = 0.5641$$

$$q_1 \cdot k_3 = 0.1155 + 0.0004 + 0.2142 + 0.3168 = 0.6469$$

Step 2. Divide by scaling factor $\sqrt{d_k} = 2.0$:

$$s_1 = 0.1682, \quad s_2 = 0.2821, \quad s_3 = 0.3235$$

Step 3. Apply Softmax to compute attention probabilities α_{1j} :

$$\exp(s_1) = 1.1832, \quad \exp(s_2) = 1.3259, \quad \exp(s_3) = 1.3820$$

$$\text{Sum} = 1.1832 + 1.3259 + 1.3820 = 3.8911$$

$$\alpha_{11} = 0.3041, \quad \alpha_{12} = 0.3407, \quad \alpha_{13} = 0.3552$$

2 Positional Encoding

Since Self-Attention is order-agnostic, positional embeddings inject word sequence position into token vectors.

The Transformer uses fixed sinusoidal functions for position pos and dimension i :

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

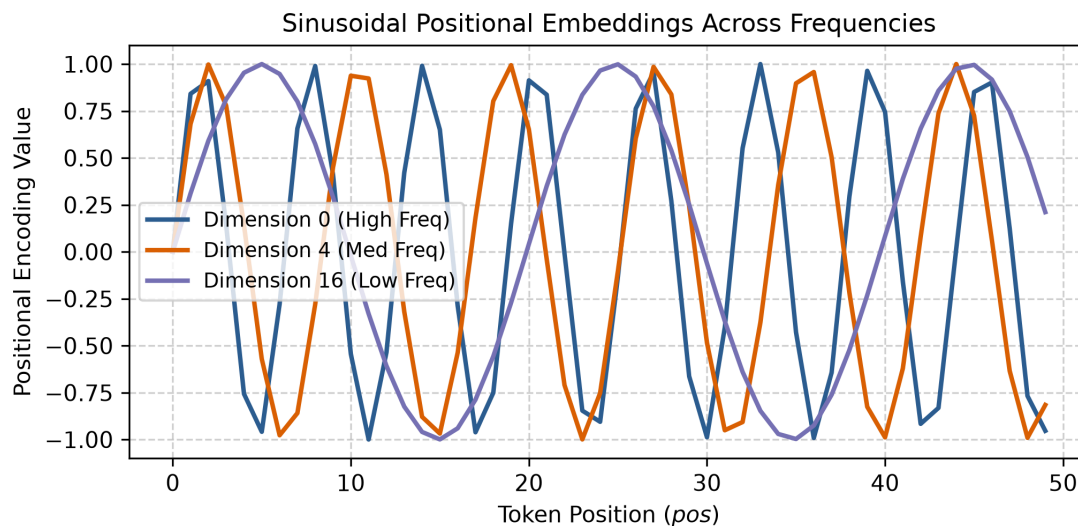


Figure 1: Sinusoidal positional encoding values across token positions for different dimension frequencies.

♣ Why Sinusoids?

Sinusoidal encodings allow the model to learn relative positions easily via linear transformations.

3 Normalization: LayerNorm vs BatchNorm

Layer Normalization normalizes across feature dimensions per sample, making it ideal for variable-length sequences.

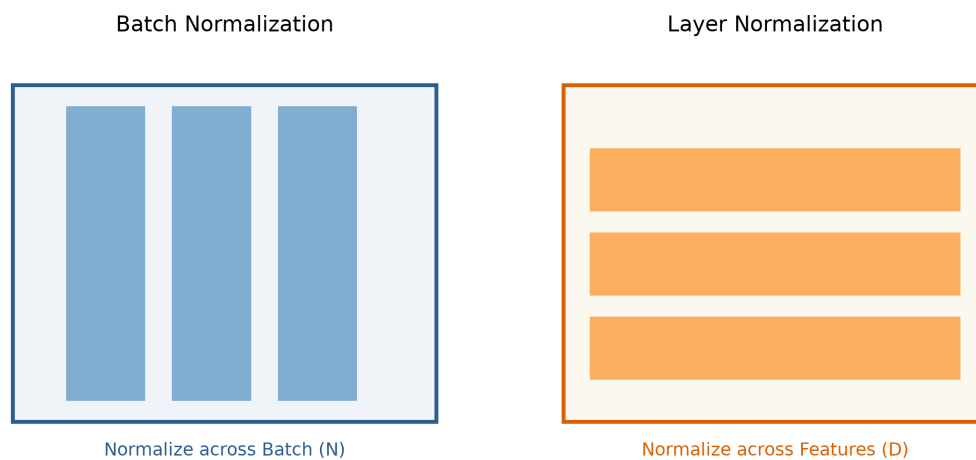


Figure 2: Comparison of Batch Normalization (across batch N) vs Layer Normalization (across features D).

✓ LayerNorm vs BatchNorm

Property	Batch Normalization (BatchNorm)	Layer Normalization (LayerNorm)
Axis	Normalizes across Batch dimension N	Normalizes across Feature dimension D
Batch Dependency	Depends heavily on batch size	Independent of batch size (works for $N = 1$)
NLP Suitability	Poor (struggles with variable lengths)	Superior (standard in Transformers)

4 BERT & Pre-training Paradigms

BERT pre-trains bidirectional encoder representations using Masked Language Modeling and Next Sentence Prediction.

BERT (Bidirectional Encoder Representations from Transformers) uses only the Transformer Encoder stack. Pre-training tasks include: 1. **Masked Language Model (MLM)**: Mask 15% of input tokens and predict them. 2. **Next Sentence Prediction (NSP)**: Predict whether Sentence B follows Sentence A.

📎 BERT Variants & Specialized Models

- **BERT Base**: 12 layers, 768 hidden size, 12 heads, 110M parameters.
- **BERT Large**: 24 layers, 1024 hidden size, 16 heads, 340M parameters.
- **patentBERT / docBERT**: Fine-tuned for patent classification and document handling.
- **TinyBERT**: Compact student model distilled from teacher BERT for fast edge inference.

5 Self-Test & Pocket Recap

Test your knowledge on Transformers, positional encodings, and BERT variants.

Self-Test Questions

1. **Question:** Why is scaling by $\frac{1}{\sqrt{d_k}}$ critical in dot-product attention?

Answer: For large values of d_k , the dot product grows large in magnitude, pushing softmax into regions with extremely small gradients. Dividing by $\sqrt{d_k}$ keeps variance at 1.0 and prevents gradient vanishing.

2. **Question:** How does BERT differ from traditional decoder-only language models like GPT?

Answer: BERT uses bidirectional attention (Encoder-only) to look at left and right context simultaneously, whereas GPT uses masked unidirectional attention (Decoder-only) to predict the next token.